

- RAG Learning

1.3 Exchanging Data

Computer Science — OCR — Component 01 (Computer Systems)

1.3.1 Compression, Encryption and Hashing

Why Compress Data?

Compression reduces the number of bits needed to represent a file, so that it takes less storage on disk and less bandwidth on a network link. The trade-off is CPU time: encoding and decoding cost cycles, so compression is worthwhile only when the savings in storage or transmission outweigh the processing overhead. Compression underpins modern computing — video streaming, ZIP archives, HTTPS content-encoding, mobile apps and backups all rely on it.

Compression is the process of encoding information using fewer bits than the original representation, either by removing redundancy (lossless) or by discarding perceptually insignificant data (lossy).

Lossy vs Lossless Compression

Feature	Lossy	Lossless
Reconstruction	Approximate — original cannot be recovered exactly	Exact — original recovered bit-for-bit after decompression
Compression ratio	High (often 10× or more)	Modest (typically 2–5×)
Typical data types	Images, audio, video (human perception masks small errors)	Text, source code, executables, spreadsheets, databases

Example formats	JPEG, MP3, AAC, MPEG-4, WebP	PNG, FLAC, ZIP, GZIP, Brotli
Suitable when...	Some quality loss is acceptable to save space	Every bit must be preserved (legal documents, program code)

Exam Tip: Never suggest lossy compression for text, code or any file where a single wrong bit would be catastrophic. A corrupted program simply will not run. Lossy compression relies on the fact that the human ear and eye cannot perceive every detail — there is no equivalent tolerance for machine-readable data.

Run-Length Encoding (RLE)

Run-Length Encoding is a lossless technique that replaces consecutive runs of the same value with a single (value, count) pair. For the string `AAAAABBBCCDAA` RLE produces `5A3B2C1D2A`.

RLE works extremely well when data has long runs of repeated symbols — for example simple bitmap icons, fax documents or artificial test data. It performs badly (can even *enlarge* files) when runs are short or absent, e.g. natural-language text where the same letter rarely repeats more than two or three times in a row.

Worked example — compressing a black-and-white bitmap row: a single row of a scanned fax document might be represented as `WWWWWWBWBWWWWWWWWWWB BBBB BBBB BBBB`. The naive representation uses 33 characters (or 33 bits). RLE encodes it as `7W 2B 12W 12B`, needing only 4 (count, value) pairs — roughly 4 bytes, a saving of over 70%. For a full page with long horizontal runs of white paper, the savings are dramatic, which is why fax machines historically used a form of RLE.

Dictionary-Based Compression (LZ77/LZW)

Dictionary-based compression replaces frequently occurring substrings with shorter codes that reference a dictionary. LZ77 uses a sliding window of recent data and emits

(offset, length, next-character) triples whenever it spots a repeat. The dictionary is rebuilt dynamically as decompression proceeds, so it does not need to be transmitted.

LZ77 and its derivatives (LZSS, DEFLATE which combines LZ77 with Huffman) are the basis of ZIP, GZIP and PNG. LZW — used historically in GIF and UNIX `compress` — maintains an explicit dictionary where each new substring seen is added with a new numeric code.

Worked example — LZ77 on "the cat sat on the mat, the rat sat on the mat": as the encoder reaches the second "the", it notices the earlier occurrence and replaces it with a back-reference pointing 15 characters back, length 4 ("the "). When it reaches "sat on the mat" at the end, a single long back-reference can replace 14 characters with one (offset, length) pair. The longer and more repetitive the source, the bigger the savings — which is why logs, natural-language text and HTML pages compress so well.

Huffman Coding

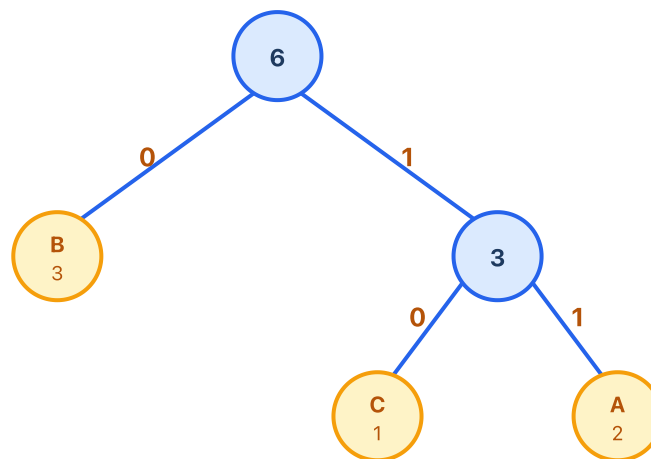
Huffman coding is a lossless algorithm that assigns variable-length binary codes to symbols based on frequency: the most common symbols get the shortest codes, and no code is a prefix of another (the **prefix property**), so a stream of concatenated codes can be decoded unambiguously.

The algorithm builds a binary tree bottom-up:

1. Count the frequency of each symbol in the source.
2. Create a leaf node for each symbol and place all leaves in a priority queue (lowest frequency first).
3. Remove the two lowest-frequency nodes, combine them under a new parent whose frequency is their sum, and reinsert the parent.
4. Repeat until one node remains — the root of the Huffman tree.
5. Labels on the tree: traversing left adds a `0` bit, right adds a `1`. The code for each symbol is the path from root to that leaf.

Worked example: encode the string `ABABBC`. Frequencies: `A` =2, `B` =3, `C` =1. Combine the two lowest (`A` =2 and `C` =1) into a subtree of weight 3, then combine that subtree with `B` =3 to form the root (weight 6).

Huffman Tree for "ABABBC"



Reading the tree: B = 0, C = 10, A = 11. Encoding ABABBC gives 11 0 11 0 0 10 = 110110010 (9 bits). A fixed-length representation at 2 bits per symbol would need 12 bits, so Huffman saves 25% on this tiny example. Real files achieve modest savings (10–40%) and Huffman is almost always combined with a dictionary stage (DEFLATE/GZIP).

Exam Tip: In exam questions you may be asked to decode a Huffman bitstream given the tree, to build the tree from frequencies, or to calculate bits saved vs fixed-length encoding. Always show workings: frequencies, the merging steps, and the final codes assigned to each symbol.

Symmetric Encryption

Symmetric encryption uses a **single shared secret key** for both encryption and decryption. Both parties must already possess the key before secure communication can begin.

Symmetric ciphers such as **AES (Advanced Encryption Standard)** and the older DES/3DES are very fast because they are designed to be implemented efficiently in hardware (AES-NI instructions on modern CPUs). They scale well to large volumes of data, which is why HTTPS uses symmetric encryption (AES-GCM or ChaCha20-Poly1305) for the bulk of the session after the initial handshake.

The fundamental weakness is the **key distribution problem**: the shared key itself must be exchanged securely. Sending the key over the same insecure channel would defeat the whole scheme.

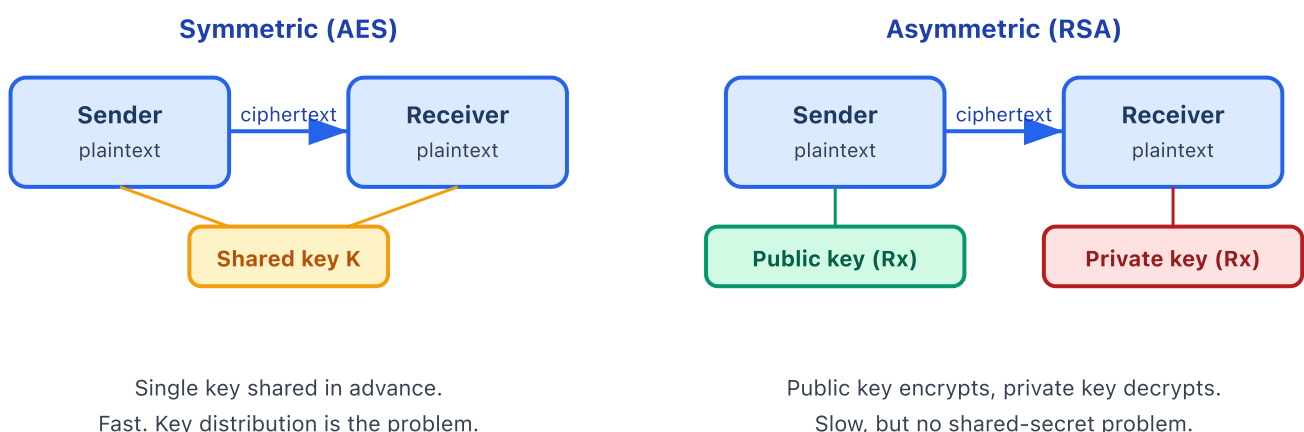
Asymmetric Encryption

Asymmetric (public-key) encryption uses a **pair of mathematically linked keys**: a **public key** that can be freely published, and a **private key** that must be kept secret. Data encrypted with the public key can only be decrypted with the corresponding private key (and vice versa).

The canonical algorithm is **RSA**, whose security rests on the difficulty of factoring the product of two very large primes: given $n = p \times q$ for 2048-bit primes, no known classical algorithm can recover p and q in reasonable time. Modern systems increasingly favour **elliptic-curve cryptography (ECC)** which achieves equivalent strength at shorter key lengths.

Asymmetric encryption solves the key distribution problem: anyone can encrypt a message to you using your public key, but only your private key decrypts it. In practice asymmetric encryption is slow (hundreds of times slower than symmetric), so it is used to **exchange a symmetric session key**, after which the parties switch to AES. This is the pattern used in TLS, PGP and Signal.

Symmetric vs Asymmetric Encryption



Hybrid approach (HTTPS/TLS)

Asymmetric crypto exchanges a symmetric session key → then AES does the bulk work.

Digital Signatures

Reversing the roles of the key pair gives a **digital signature**. The sender hashes the message, encrypts the hash with their **private** key and sends both message and signature. The receiver decrypts the signature with the sender's **public** key and compares the result with a freshly computed hash. If they match, the message is authentic (only the holder of the private key could have signed it) and untampered (any change would alter the hash).

Digital signatures provide three guarantees at once:

- **Authenticity** — the message really came from the claimed sender.
- **Integrity** — the message has not been altered in transit.
- **Non-repudiation** — the sender cannot later plausibly deny having sent it, because only they hold the private key.

This is why HTTPS certificates (issued by a Certificate Authority) are digitally signed: your browser trusts `www.ocr.org.uk` because it can verify the CA's signature on OCR's certificate against the CA's public key, which is built into the operating system.

Hashing

A **hash function** is a one-way function that maps an input of arbitrary length to a fixed-length output (the **hash** or **digest**). A **cryptographic** hash additionally satisfies: (i) it is computationally infeasible to reverse; (ii) a small change in input produces a completely different output (avalanche effect); (iii) it is computationally infeasible to find two inputs with the same hash (collision resistance).

Common algorithms include MD5 and SHA-1 (both now considered broken), SHA-256, SHA-3 and BLAKE3. Non-cryptographic hashes such as the ones used in hash tables (e.g. FNV, MurmurHash) prioritise speed and even distribution over security.

The avalanche effect in practice: hashing `"password"` with SHA-256 gives the digest beginning `5e884898...`. Hashing `"Password"` (one bit of difference in ASCII) gives `e7cf3ef4...`. The outputs share no recognisable structure, which is what makes hashes useful as fingerprints: any tampering, no matter how small, produces an entirely different digest.

Uses of hashing:

- **Password storage** — only the hash is stored. When a user logs in, the entered password is hashed and compared. A stolen database gives attackers hashes, not passwords.
- **Salting** — a random per-user **salt** is concatenated to the password before hashing, so two users with identical passwords store different hashes, defeating rainbow-table attacks.
- **File integrity** — publishing a SHA-256 digest alongside a downloaded file lets users verify it has not been tampered with.
- **Hash tables** — the hash of a key picks a bucket (covered in 2.2). A **collision** is when two distinct inputs hash to the same bucket; handled by chaining or open addressing.
- **Digital signatures / blockchains** — hashes chain blocks and authenticate messages.

Forward reference to 2.2: a hash table uses a hash function to map a key directly to an array index, giving average $O(1)$ insert and lookup. If two keys hash to the same index (a **collision**), the table resolves it via *separate chaining* (a linked list at each bucket) or *open addressing* (probing the next bucket). For this to work well, the hash function must spread keys evenly; a poor hash degrades lookup to $O(n)$. Password hashing uses a slow, salted cryptographic hash (bcrypt, scrypt, Argon2) deliberately — the opposite of what you want for a hash table — to make offline brute-force attacks expensive.

Exam Tip: Encryption and hashing are frequently confused. **Encryption is two-way:** the ciphertext can be decrypted back to plaintext using a key. **Hashing is one-way:** there is no decrypt operation. You store passwords as hashes (not encrypted) precisely because even a full database leak should not reveal the originals.

1.3.2 Databases

Relational Database Fundamentals

A **relational database** organises data into **tables (relations)** made up of **records (rows/tuples)**, each containing the same set of **fields (columns/attributes)**. Tables are linked through common fields rather than by physical pointers.

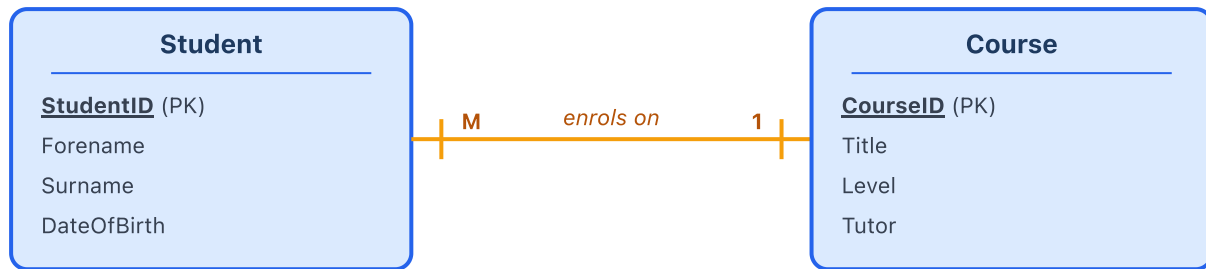
Term	Meaning
Entity	A real-world thing about which data is stored (Student, Book, Order). Each entity normally becomes a table.
Attribute	A property of an entity (a column in the table).
Record	One row, i.e. one instance of an entity.
Primary key	An attribute (or combination) that uniquely identifies each record in a table. Must be unique and not NULL.
Foreign key	An attribute in one table that refers to the primary key of another table. It enforces a link between the two.
Composite key	A primary key that consists of two or more attributes, used when no single attribute is unique.
Secondary key	An indexed attribute used to speed up searches but not necessarily unique, e.g. Surname.

Entity-Relationship (ER) Diagrams

An ER diagram models the logical structure of a database before any SQL is written. Entities are drawn as rectangles, attributes as ovals (or listed inside the rectangle), and relationships as a line between two entities, labelled with the nature of the link and its cardinality.

Cardinalities are usually drawn as **1:1** (one-to-one), **1:N** (one-to-many) or **M:N** (many-to-many). A many-to-many relationship cannot be implemented directly in a relational database — it must be broken down using a **linking (junction) table** that contains foreign keys to both sides.

Entity-Relationship Diagram: Student — Course



This is a 1:M relationship: one Course has many Students; each Student is on exactly one Course.

Normalisation

Normalisation is the process of structuring a relational database to reduce data redundancy and improve data integrity. Data is broken down into separate tables so that each fact is stored in exactly one place.

Normalisation is carried out in stages called **normal forms**. Each stage builds on the last.

Redundant data is a problem for three reasons: it wastes storage, it forces every update to happen in multiple places (an **update anomaly**), and it risks inconsistency if one copy is updated and the others forgotten. Normalisation eliminates these anomalies systematically.

First Normal Form (1NF): every field contains only an *atomic* value (no lists, no repeating groups); each record is unique (has a primary key); entries in a column are all of the same data type.

Second Normal Form (2NF): the database is in 1NF *and* every non-key attribute depends on the *whole* of a composite primary key, not just part of it. If there is a partial dependency, split it into a new table.

Third Normal Form (3NF): the database is in 2NF *and* there are no **transitive dependencies** — non-key attributes must depend only on the primary key, not on other non-key attributes.

A popular mnemonic is "Each attribute must depend on **the key, the whole key, and nothing but the key**, so help me Codd."

Normalisation: UNF → 1NF → 2NF → 3NF

Unnormalised: repeating group of subjects in one field

StudentID	Name	Subjects	TutorName, TutorRoom
S01	Alice	Maths, Physics, CS	Mr Brown, R12

1NF

Atomic values; composite PK (StudentID, Subject)

SID	Name	Subj	Tutor/Rm
S01	Alice	Maths	Brown/R12
S01	Alice	Phys	Brown/R12
S01	Alice	CS	Brown/R12
S02	Ben	Maths	Brown/R12

2NF

Remove partial dependencies; two tables

SID	Name/Tutor/Rm
S01	Alice/Brown/R12
S02	Ben/Brown/R12

SID	Subject
S01	Maths
S01	Phys
S01	CS

3NF

Remove transitive deps; separate Tutor table

SID	Name, Tutor	TID	Tutor, Room
S01	Alice, T1	T1	Brown, R12
S02	Ben, T1		

Each stage removes a specific kind of redundancy:

1NF: remove repeating groups by making each row hold one subject.

2NF: Student.Name depended only on SID, not on the whole (SID, Subject) composite key — split it out.

3NF: Room depended on Tutor (not on SID) — a transitive dependency, so a separate Tutor table is created.

Exam Tip: Students often confuse normalisation with compression. They are unrelated.

Normalisation removes logical redundancy in a relational database so each fact is stored once. **Compression** is a separate, lower-level process that reduces the physical number of bits needed to represent a file on disk or transmit it over a network.

Update, Insert and Delete Anomalies

Poorly normalised databases are prone to three kinds of anomaly, each of which is removed by progressing to a higher normal form.

Anomaly	What goes wrong	Example
Update anomaly	The same fact is stored in many rows, so an update must find and change them all or else leave the database inconsistent.	A tutor's room number is duplicated against every student they teach. If the tutor changes room, every row must be updated.
Insertion anomaly	You cannot record a fact because some required related fact is missing.	You cannot add a new course until at least one student has enrolled,

		because the course details live inside the student table.
Deletion anomaly	Deleting one record accidentally destroys unrelated information.	Removing the last student of a particular course also removes the only record of that course's existence.

Referential Integrity

Referential integrity is the guarantee that a foreign key value always matches an existing primary key in the referenced table (or is NULL). A customer order cannot reference a customer who does not exist.

The database enforces this on INSERT and UPDATE. When a record that is referenced elsewhere is deleted or updated, the DBMS must decide what to do. The policies are:

- **CASCADE** — propagate the change: deleting a customer deletes all their orders; updating a CustomerID updates it in the orders table.
- **SET NULL** — set the foreign key to NULL.
- **RESTRICT / NO ACTION** — forbid the operation if any referencing rows exist.

SQL: Structured Query Language

SQL is the declarative language used to define and manipulate relational data. The most important statements at A-level are `SELECT`, `INSERT`, `UPDATE`, `DELETE`, and the clauses that modify them: `WHERE`, `ORDER BY`, `GROUP BY`, `HAVING`, `JOIN`.

```
-- Select particular fields
SELECT Forename, Surname, DateOfBirth
FROM Student
WHERE YearGroup = 13
ORDER BY Surname ASC;

-- Insert a new record
INSERT INTO Student (StudentID, Forename, Surname, YearGroup)
VALUES ('S042', 'Priya', 'Patel', 12);
```

```
-- Update
UPDATE Student
SET     YearGroup = 13
WHERE  YearGroup = 12;

-- Delete
DELETE FROM Student
WHERE  StudentID = 'S042';

-- Aggregate + HAVING: how many students per year group, only
-- listing year groups that have more than 100 students?
SELECT  YearGroup, COUNT(*) AS NumStudents
FROM    Student
GROUP BY YearGroup
HAVING  COUNT(*) > 100;

-- Inner join: pull student name and course title together
SELECT  S.Forename, S.Surname, C.Title
FROM    Student S
JOIN    Course  C ON S.CourseID = C.CourseID
WHERE   C.Level = 'A-Level'
ORDER BY S.Surname;
```

WHERE filters *rows* before grouping. **HAVING** filters *groups* after **GROUP BY** has done its work. Mixing them up is a classic exam error.

JOIN types

Join	Meaning
INNER JOIN	Rows appearing in both tables where the ON condition matches. Non-matching rows are omitted.
LEFT (OUTER) JOIN	All rows from the left table; matching rows from the right if any, otherwise NULLs. Useful for “list all students including those not on a course”.
RIGHT (OUTER) JOIN	Mirror image — all rows from the right table.

FULL OUTER JOIN	All rows from both tables; missing sides filled with NULLs.
CROSS JOIN	Cartesian product — every row in A paired with every row in B. Rarely what you want; often a mistake.

```
-- LEFT JOIN: include students who are not yet assigned a course
SELECT  S.Forename, S.Surname, C.Title
FROM    Student S
LEFT JOIN Course  C ON S.CourseID = C.CourseID
ORDER BY S.Surname;
```

Exam Tip: Remember the SQL keyword order: `SELECT ... FROM ... JOIN ... WHERE ... GROUP BY ... HAVING ... ORDER BY`. The DBMS executes them roughly in the order `FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY`.

Transaction Processing and ACID

A **transaction** is a logical unit of work consisting of one or more database operations that must all succeed or all fail together. Classic example: transferring £100 from account A to account B requires two updates (debit A, credit B), and the database must never be left in a state where one half has happened without the other.

The four ACID properties that any transactional DBMS must uphold:

- **Atomicity** — the transaction is all-or-nothing. If any step fails, the whole transaction is **rolled back** to the state before it started.
- **Consistency** — the transaction takes the database from one valid state to another; all constraints (primary keys, foreign keys, CHECK rules) hold before and after.
- **Isolation** — concurrent transactions must not see each other's partial updates. The result is as if they had run one at a time.
- **Durability** — once a transaction has been committed, its effects survive subsequent crashes or power failures, because they have been written to persistent storage (typically via a transaction log).

log).

Worked example — bank transfer:

```
BEGIN TRANSACTION;  
    UPDATE Account SET Balance = Balance - 100 WHERE AccNo = 'A';  
    UPDATE Account SET Balance = Balance + 100 WHERE AccNo = 'B';  
COMMIT;
```

Without ACID: a crash between the two `UPDATE` s would leave the system missing £100. With ACID, the partially completed work is rolled back on restart and the user sees the original balances.

Record Locking and Concurrency

Record locking prevents two users from updating the same record simultaneously. When one transaction locks a record, others attempting to modify it are queued until the lock is released. Locks can be **shared** (read) or **exclusive** (write). Two transactions waiting on each other's locks forever form a **deadlock**, which the DBMS resolves by aborting one transaction.

Redundancy in this context means keeping more than one copy of the database (mirroring, replication, RAID) so that hardware failure does not cause data loss — part of how *durability* is achieved in practice.

Worked example — concurrency without locking: suppose two cashpoints simultaneously read the same account balance of £200. Customer X withdraws £150, Customer Y withdraws £100. Both machines compute a new balance based on the initial £200 they each read (£50 and £100 respectively) and each writes its result back. Whichever transaction commits last “wins”, and the bank loses £100 or £150. This is called the **lost update** problem. Record locking forces one transaction to wait until the other has committed, after which it reads the updated balance and proceeds correctly.

Capturing, Selecting, Managing and Exchanging Data

- **Online forms** — structured input validated at the point of capture (required fields, format checks).
- **Bar codes** — optical patterns on products; scanned to retrieve a product record (stock control, supermarket tills).

- **QR codes** — 2D evolution of barcodes; carry URLs, payment data or structured payloads.
- **RFID tags** — radio-frequency chips read wirelessly at short range. Used for contactless cards, travel passes, warehouse stock.
- **OCR** — optical character recognition for reading printed text into a database.
- **OMR** — optical mark recognition for multiple-choice answer sheets.
- **Data exchange formats** — CSV, XML and JSON for passing data between systems. Mainstream web APIs use JSON.

Automated capture methods matter because they reduce transcription errors and increase throughput. A supermarket till scanning a barcode takes milliseconds and cannot make a typo; a human keying the same number takes seconds and may get one digit wrong. Validating data at the point of capture (check digits on barcodes, format rules on online forms) catches errors before they enter the database, where they are expensive to fix.

1.3.3 Networks

Types and Characteristics

Type	Scope	Typical use
PAN (Personal Area Network)	A few metres	Bluetooth earbuds, phone–laptop tether
LAN (Local Area Network)	A single site (home, office, school)	Wi-Fi, office file server, local printers
WAN (Wide Area Network)	Across cities, countries or continents	Corporate site-to-site links, the Internet

Bandwidth is the *capacity* of a link (bits per second). **Latency** is the *delay* a single bit experiences travelling end-to-end. **Throughput** is the *achieved* data rate after overheads, losses and retransmissions. A link can have high bandwidth but poor throughput if latency and loss are high.

Protocols and Why They Matter

A **protocol** is an agreed set of rules governing communication: format, timing, error handling, sequencing. Without shared protocols, machines from different vendors could not interoperate.

Real-world example of the bandwidth/latency distinction: a satellite broadband link may offer 100 Mbps of bandwidth but adds 600 ms of latency because of the 36,000 km round-trip to geostationary orbit. Bulk downloads still complete quickly, but interactive applications (video calls, online gaming, SSH) feel laggy because each request must wait a full round-trip. A fibre link at 50 Mbps and 5 ms latency would feel far more responsive despite its lower “headline” speed.

Data Transmission Modes

- **Simplex** — data flows in one direction only (TV broadcast, old serial printers).
- **Half-duplex** — data flows in both directions but not simultaneously (walkie-talkie).
- **Full-duplex** — data flows in both directions at the same time (modern Ethernet, Wi-Fi).

Within a LAN, devices may be connected via a **switch**, which learns the MAC addresses of each attached device and forwards frames only to the correct port, or a (now mostly historical) **hub** which broadcast every frame to every port. Routers operate at the Internet/Network layer, forwarding packets between different networks on the basis of IP addresses.

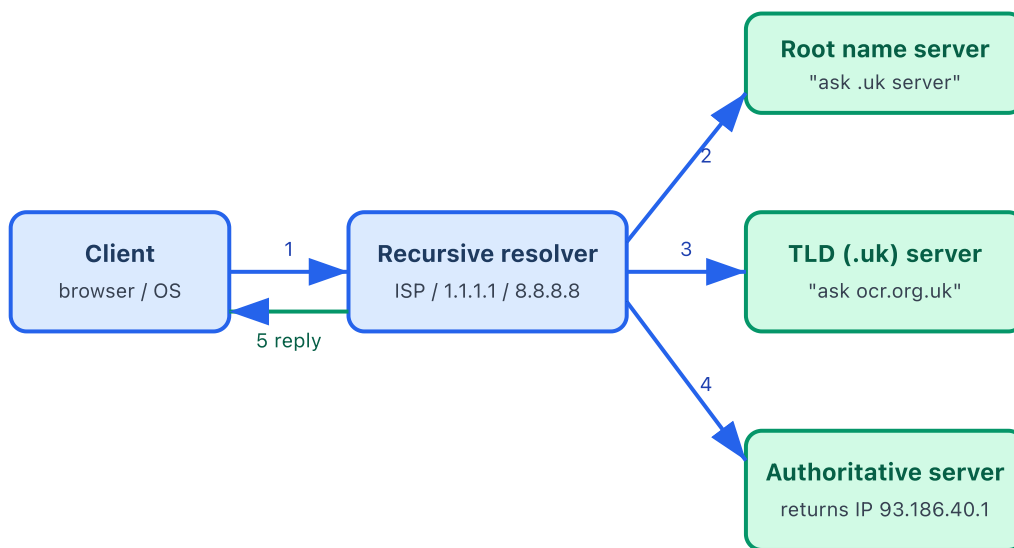
The Internet, ISPs and DNS

The Internet is a global WAN formed by interconnecting thousands of smaller networks (tier-1, tier-2 and tier-3 ISPs) via **routers** and **Internet Exchange Points**. Home users connect through a consumer ISP who provides a public IP address (often dynamically) and a route up into the global routing mesh.

The **Domain Name System (DNS)** is the distributed database that maps human-friendly domain names (e.g. `www.ocr.org.uk`) to IP addresses. It is organised as a tree: root → top-level domain (`.uk` , `.com`) → second-level domain (`ocr.org.uk`) → host (`www`).

A typical DNS lookup from a user's browser proceeds as follows:

DNS Resolution for `www.ocr.org.uk`



1. Client asks its configured **recursive resolver** (typically at the ISP).
2. If not cached, the resolver queries a **root** server, which replies with the address of the `.uk` TLD server.
3. The resolver queries the `.uk` TLD server, which replies with the authoritative server for `ocr.org.uk`.
4. The resolver queries the **authoritative** name server for the A record, which returns the IP address.
5. The resolver caches the answer (respecting the record's TTL) and returns it to the client.

IP Addresses, NAT and URLs

IPv4 addresses are 32 bits (four 8-bit octets, e.g. `192.168.0.12`) giving ~4.3 billion possible addresses — long exhausted. **IPv6** uses 128 bits (e.g. `2001:db8::1`) giving 3.4×10^{38} addresses, enough for every device imaginable.

NAT (Network Address Translation) lets many devices on a private LAN (addresses like `192.168.x.x`) share a single public IPv4 address. The router rewrites source addresses and ports for outbound traffic, and keeps a table so replies are returned to the correct internal device. NAT prolonged IPv4 but breaks some peer-to-peer protocols.

Subnetting splits a large IP block into smaller networks by extending the number of bits reserved for the network portion of the address, e.g. `192.168.1.0/24` reserves 24 bits for network and

leaves 8 bits (256 addresses) for hosts.

A **URL** has the form `scheme://host:port/path?query#fragment` , for example

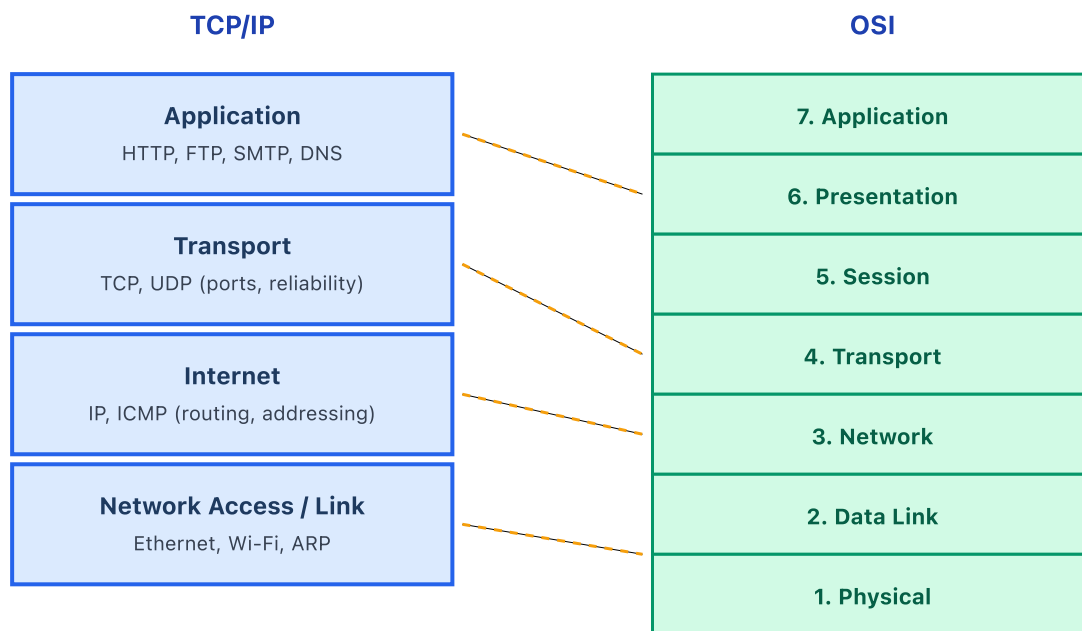
`https://www.ocr.org.uk:443/qualifications/as-and-a-level/computer-science-h446/?year=2024#overview` .

URL Component	Example	Role
Scheme	<code>https</code>	Protocol used to fetch the resource.
Host	<code>www.ocr.org.uk</code>	Domain name resolved by DNS.
Port	<code>443</code>	Logical endpoint on the host (omitted if default).
Path	<code>/qualifications/.../h446/</code>	Identifies the resource on the server.
Query	<code>?year=2024</code>	Parameters passed to server-side code.
Fragment	<code>#overview</code>	Client-side anchor within the document.

TCP/IP and OSI Models

The **OSI 7-layer model** is a conceptual reference; the **TCP/IP 4-layer model** is what the Internet actually implements. Each layer offers a service to the layer above using the layer below, and communicates with its peer on the other host by adding a header (encapsulation).

TCP/IP (4-layer) vs OSI (7-layer)



TCP/IP Application $\approx$ OSI 5–7, TCP/IP Transport $\approx$ OSI 4,
TCP/IP Internet $\approx$ OSI 3, TCP/IP Network Access $\approx$ OSI 1–2.

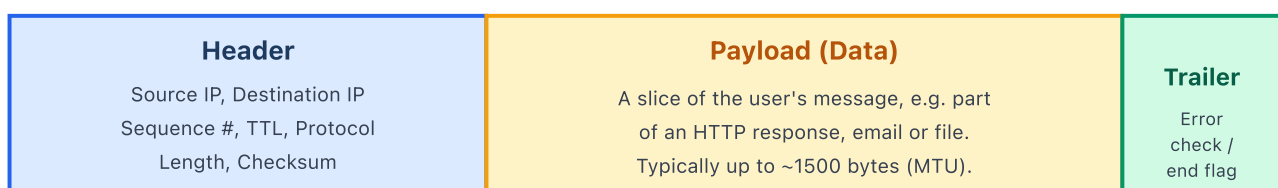
Packet Switching vs Circuit Switching

Circuit switching sets up a dedicated end-to-end path for the whole call (traditional telephone network). The path is guaranteed but wasteful during silences.

Packet switching breaks messages into small **packets**, each routed independently across the network and reassembled at the destination. Packets may take different routes and arrive out of order. The model is robust (automatic re-routing around failures), efficient (bandwidth shared dynamically) and scalable — the foundation of the Internet.

Every packet contains three parts: a **header**, a **payload** and (optionally) a **trailer**.

Structure of a Network Packet



Routers look only at the header to decide where next to forward the packet.

Common Protocols

Protocol	Layer	Purpose
HTTP / HTTPS	Application	Web pages; HTTPS wraps HTTP in TLS for confidentiality and integrity.
FTP	Application	File transfer between client and server.
SMTP	Application	Sends outgoing email between mail servers.
POP3	Application	Downloads messages from a mail server to a single client and usually deletes the server copy.
IMAP	Application	Keeps mail on the server; supports folders and multi-device sync.
TCP	Transport	Reliable, connection-oriented, ordered byte stream with retransmission.
UDP	Transport	Connectionless, fire-and-forget datagrams. Low overhead. Used for DNS, VoIP, live video, online games.
IP	Internet	Addresses hosts and routes packets between networks.
TLS/SSL	Session-ish	Provides confidentiality, integrity and authentication for protocols like HTTPS.

TLS handshake (simplified): client sends "ClientHello" with supported ciphers; server replies with its X.509 certificate and chosen cipher; client verifies the certificate chain against trusted root CAs; client and server perform a key-exchange (RSA or Diffie-Hellman) to derive a shared symmetric session key; the rest of the session is encrypted with AES (or similar).

TCP vs UDP in context: if you are downloading a spreadsheet, losing a single byte would corrupt the file, so TCP is ideal — it will retransmit anything lost and deliver bytes in order. If you are on a video call, a lost frame is better ignored than redelivered a second late, so UDP is preferred: the application handles loss by skipping or concealing. TCP's three-way handshake (SYN, SYN-ACK, ACK) establishes a connection and synchronises sequence numbers before any data flows.

Encapsulation as Data Descends the Stack

When an application sends data, each layer adds its own header (and possibly trailer) before passing the unit down. At the destination, each layer strips its corresponding header before passing the payload up. This nesting is called **encapsulation**.

Layer	Unit of data	Added by the layer
Application	Message (e.g. HTTP request)	Application-specific fields (URL, headers)
Transport	Segment (TCP) / Datagram (UDP)	Source + destination port , sequence #, checksum
Internet	Packet	Source + destination IP , TTL
Network Access	Frame	Source + destination MAC , CRC trailer

Exam Tip: Ports identify *applications* on a host (80 for HTTP, 443 for HTTPS, 25 for SMTP, 53 for DNS). IP addresses identify *hosts* on a network. MAC addresses identify *network interfaces* on a local link. An examiner may ask which part of the stack each is associated with.

Client-Server vs Peer-to-Peer

Aspect	Client-server	Peer-to-peer (P2P)
Structure	Central server provides services to many clients	All nodes are equal; each is simultaneously client and server
Examples	The Web, email, databases, online games (MMO)	BitTorrent, blockchains, Skype (historically), some IoT mesh networks
Reliability	Single point of failure at the server; mitigated by replication	Highly resilient — no single node is critical

Scalability	Server capacity must scale with users	Scales with participants (more peers = more capacity)
Security / control	Centralised administration, easier to secure	Harder to police, easier to abuse (piracy, malware)
Cost	High server + hosting costs	Very low infrastructure cost

Peer-to-Peer: How BitTorrent Actually Works

A concrete case of P2P architecture: a BitTorrent user downloading a file does not contact a single central server. Instead, a small `.torrent` metadata file or magnet link describes the content and lists a **tracker** (or uses DHT, a distributed hash table, as a decentralised replacement). The tracker returns a list of peers already holding chunks of the file. The client simultaneously downloads different chunks from different peers and, as soon as it has any chunk, starts uploading it to others. The more popular the file, the faster it downloads — the opposite of a central server, which gets overloaded by popularity. The cost to the content publisher is near zero: they only need enough initial seed bandwidth to push each chunk out once, after which the swarm distributes it.

Firewalls

A **firewall** is a hardware or software device that filters traffic between networks according to rules. Its job is to let legitimate traffic through while blocking unwanted or malicious traffic.

- **Packet filtering** — inspects each packet's header (source/destination IP and port, protocol) and accepts or drops it. Stateless and fast.
- **Stateful inspection** — tracks active connections so it can recognise that an inbound packet is a reply to a request the user initiated. More effective against spoofing.
- **Proxy server** — sits between clients and the outside network, forwarding requests on the clients' behalf. Can cache content, log usage and enforce content policies.

Wireless Networks and Security

- **Wi-Fi** (IEEE 802.11) is the dominant wireless LAN standard. Versions: 802.11n (Wi-Fi 4), ac (Wi-Fi 5), ax (Wi-Fi 6/6E), be (Wi-Fi 7).

- **SSID** — the human-readable network name broadcast by the access point. Hiding SSID broadcasts is weak security because the name still appears when clients connect.
- **MAC filtering** — only devices whose MAC addresses appear in an allow-list may join. Trivially bypassed by spoofing MAC addresses.
- **WPA** replaced the broken WEP in 2003. **WPA2** uses AES-CCMP and has been the standard since 2006. **WPA3** (2018) adds individualised data encryption (SAE) and protects against offline dictionary attacks on the handshake.

Wireless networks are inherently broadcast media — anyone within range can receive the radio signal, so encryption is not optional. A home router typically offers WPA2-PSK (pre-shared key) where all devices share one passphrase; an enterprise deployment uses WPA2/3-Enterprise with 802.1X authentication so each user has individual credentials that can be revoked.

Exam Tip: Hiding an SSID and using MAC filtering are not substitutes for strong encryption. Examiners expect you to recognise these as weak, "security by obscurity" measures and recommend WPA2 or WPA3 as the real defence.

1.3.4 Web Technologies

HTML, CSS and JavaScript

The three client-side web technologies have complementary roles, summarised as **structure**, **style**, **behaviour**.

Technology	Role	Runs on
HTML (HyperText Markup Language)	Structure and semantics of a page — headings, paragraphs, links, forms, images.	Parsed by the browser into the DOM.
CSS (Cascading Style Sheets)	Presentation: colours, fonts, layout, responsiveness. Separated from content so one stylesheet can style many pages.	Applied by the browser's rendering engine.

JavaScript	Behaviour: event handlers, DOM manipulation, fetching data without reloading (AJAX / fetch).	Executed by the browser's JS engine (V8, SpiderMonkey, JavaScriptCore).
-------------------	--	---

```

<!-- HTML: structure -->
<h1 id="title">Welcome</h1>
<button id="btn">Click me</button>

/* CSS: style */
#title { color: #1e40af; font-family: sans-serif; }
#btn { padding: 8px 16px; background: #2563eb; color: #fff; }

// JavaScript: behaviour
document.getElementById('btn').addEventListener('click', () => {
    document.getElementById('title').textContent = 'Hello!';
});

```

Client-Side vs Server-Side Processing

Client-side processing (JavaScript running in the browser) gives immediate feedback, reduces round-trips and offloads work from the server. It cannot be trusted for anything security-sensitive because the user can inspect and modify the code.

Server-side processing (PHP, Node.js, Python, ASP.NET, Ruby...) runs on the web server and is authoritative. It accesses private databases, performs validation and authentication, and generates dynamic HTML. It is trusted because the user cannot see or modify it, but every request must traverse the network.

Use client-side for...	Use server-side for...
Immediate UI feedback (form field validation, hover effects)	Authoritative validation of submitted data
Reducing server load for trivial computation	Database queries
Offline functionality (PWA, service workers)	Authentication, authorisation, payments

Rich interactivity (drag-drop, animations)

Access to private APIs, secret keys

Exam Tip: A standard exam trap: "Is client-side validation enough?" **No.** Always re-validate on the server, because client-side JavaScript can be disabled or edited by the user. Client-side validation is a usability feature; server-side validation is a security requirement.

Search Engine Indexing

A **search engine** such as Google or Bing consists of three co-operating parts: a crawler, an index and a ranking engine.

- **Web crawler / spider** — an automated program that recursively fetches web pages by following hyperlinks, respecting `robots.txt` directives and polite rate limits.
- **Indexer** — processes the fetched HTML, extracts meaningful tokens and builds an **inverted index** mapping each word to the list of pages containing it. This is why a search for "OCR computer science" can return answers in milliseconds.
- **Ranker** — given a query, retrieves candidate pages from the index and orders them by relevance using hundreds of factors: keyword match, freshness, authority, mobile-friendliness, user engagement signals, and link-based measures like PageRank.

PageRank

PageRank (Brin & Page, 1998) is an algorithm that ranks web pages by the idea that a page is *important if many important pages link to it*. The rank of a page is computed iteratively from the ranks of the pages linking to it, damped by a factor d (usually 0.85) that models a random surfer occasionally jumping to a random page rather than following a link.

Informally: $PR(A) = (1 - d) + d \times \sum (PR(T_i) / C(T_i))$ summed over pages T_i that link to A , where $C(T_i)$ is the number of outbound links on T_i . The system is iterated until the values converge. Intuitively, a link from a highly ranked page transmits more "vote" than a link from an obscure one, and a page that fans out to many others passes only a small fraction of its rank to each.

PageRank is just one input to modern ranking, which also considers content quality, query intent, personalisation and machine-learned signals (RankBrain, BERT-based relevance).

Why PageRank was revolutionary: early search engines (AltaVista, Lycos) ranked results by counting keyword matches on each page, which was easy to game — a spam page could stuff keywords into hidden text. PageRank shifted the basis of trust from what a page says about itself to what *other pages* say about it by linking. A link from an established authority (a university, a newspaper, another widely cited site) confers far more rank than a link from a freshly created spam site, because those authorities themselves have high PageRank. This feedback structure makes PageRank robust to single-source manipulation.

Server-Side Scripting

Server-side scripts run on the web server in response to each request, producing HTML (or JSON, for APIs) dynamically. Common stacks:

- **PHP** — long-standing, powers WordPress and much of the older Web.
- **Node.js** — JavaScript on the server, typically with Express or Next.js.
- **Python** — Django or Flask for web apps, Jupyter for data work.
- **ASP.NET** — Microsoft's managed framework (C#, F#).
- **Ruby on Rails, Java Spring, Go** — other mainstream options.

A typical request lifecycle: browser sends an HTTP request → web server forwards to the application runtime → script queries the database, performs logic → response HTML is rendered and returned over HTTPS. Sessions, cookies and JSON Web Tokens (JWT) are used to persist identity across the stateless HTTP protocol.

Exam Tip: If a question asks "why use server-side scripting instead of just more JavaScript in the browser?", focus on three ideas: (1) access to private databases and secret keys which must never reach the client; (2) centralised business logic — one source of truth for every client, including mobile apps; (3) security — the user cannot tamper with code they cannot see.

Worked Case Studies

Case Study 0: Login and Password Storage

When you sign up for an account the server does *not* save your password. Instead, it generates a random salt, concatenates it with your password, runs the result through a slow cryptographic hash (bcrypt, Argon2) and stores the pair (salt, hash). When you log in next time, the server retrieves the salt, hashes *your* entered password with that salt, and compares the result to the stored hash. A match authenticates you. An attacker who steals the database gets only hashes — they cannot log in as any user without brute-forcing each password, and the salt ensures that pre-computed rainbow tables are useless. This is why reputable websites can never “send you your password”: they do not have it.

Case Study 1: Online Shop Checkout

A customer clicks “Buy” on an online shop. The touch-point exercises almost every concept in 1.3:

1. The browser resolves the shop's domain via **DNS** and opens a **TCP** connection to port 443.
2. A **TLS handshake** verifies the server's certificate and establishes a symmetric **AES** session key using **asymmetric** crypto.
3. The browser sends an **HTTPS** POST with order details. The payload is split into multiple **packets**, each carrying a **header** with source/destination IPs and sequence numbers, and routed independently.
4. The server-side script (**PHP**, **Node**...) queries a **relational database** inside a single **ACID transaction**: decrement stock, create an order, charge the payment processor. All three steps must succeed together or all roll back.
5. The user's password is never transmitted in the clear; only a **salted hash** is stored on the server for authentication.
6. The response HTML is returned, optionally **GZIP compressed** (LZ77 + Huffman) to save bandwidth.

Case Study 2: Streaming a Song

A music streaming service uses **lossy compression** (typically AAC) so that a 50 MB studio file shrinks to ~5 MB. Delivery uses **HTTPS** over **TCP** (for reliability) with adaptive bitrate so quality is downgraded when throughput drops. Metadata (track, artist, album art) is kept in a **relational database** indexed by track ID. Play counts are written asynchronously to avoid blocking the audio stream, illustrating the trade-off between strict ACID and high throughput.

Common Exam Pitfalls — Recap

- **Encryption vs hashing** — encryption is reversible with a key; hashing is one-way.
- **Normalisation vs compression** — normalisation is logical (removes data redundancy); compression is physical (reduces bits).
- **Bandwidth vs latency vs throughput** — bandwidth is capacity, latency is delay, throughput is actually achieved rate.
- **WHERE vs HAVING** — WHERE filters rows; HAVING filters groups after `GROUP BY`.
- **Client-side vs server-side validation** — client-side is for UX; server-side is the only real security boundary.
- **Symmetric vs asymmetric** — asymmetric solves key distribution but is slow; real systems use it only to establish a symmetric session key.
- **TCP vs UDP** — TCP reliable and ordered; UDP fast and loss-tolerant.
- **Circuit vs packet switching** — circuit reserves a dedicated path (old phones); packet shares the network dynamically (the Internet).
- **1:1, 1:M, M:N** cardinalities — M:N must be resolved with a linking table.

Quick-Recall Summary

- **Compression** reduces size: lossy for media, lossless for code/text. RLE for runs, dictionary (LZ77) for repeats, Huffman for variable-length codes.
- **Encryption**: symmetric (one key, fast, key-distribution problem) vs asymmetric (public+private keys, solves distribution, slow). TLS uses both.
- **Hashing** is one-way; used for passwords (with salt), hash tables and digital signatures.
- **Relational databases**: keys link tables. Normalise to 3NF to remove redundancy. ACID guarantees correct transactions.
- **SQL**: SELECT / INSERT / UPDATE / DELETE / JOIN / GROUP BY / HAVING.
- **Networks**: LAN/WAN/PAN; TCP/IP 4-layer vs OSI 7-layer; packet switching with header + payload + trailer.
- **DNS** resolves names through root → TLD → authoritative servers.

- **Client-server vs P2P**; firewalls (packet, stateful, proxy); Wi-Fi security WPA2/WPA3 > MAC filtering or hidden SSID.
- **Web**: HTML = structure, CSS = style, JavaScript = behaviour. Server-side for trust and databases; client-side for responsiveness.
- **Search**: crawl → index → rank. PageRank ranks by weighted incoming links.

Key Definitions At-a-Glance

Term	Definition
Lossy compression	Discards data the human eye/ear cannot easily perceive; file cannot be restored to its original.
Lossless compression	Encodes data using fewer bits so the original can be perfectly reconstructed.
Huffman coding	Variable-length prefix code where frequent symbols get shorter codes.
Symmetric encryption	Single secret key used for both encryption and decryption (e.g. AES).
Asymmetric encryption	Public/private key pair; public key encrypts, private key decrypts (e.g. RSA).
Hash function	One-way function mapping arbitrary-length input to a fixed-length digest.
Primary key	Attribute (or combination) uniquely identifying each record.
Foreign key	Attribute referring to the primary key of another table.
Normalisation	Restructuring a database to eliminate redundancy and anomalies.
ACID	Atomicity, Consistency, Isolation, Durability — the guarantees a transaction must provide.
Protocol	Agreed set of rules governing communication between systems.

DNS	Distributed system translating domain names to IP addresses.
Packet switching	Technique of splitting data into packets routed independently across a shared network.
Firewall	Network device filtering traffic according to security rules.
PageRank	Algorithm ranking pages by the weighted count of incoming links.